

NagarVaani — Project Document

Your Voice, Your City · नगरवाणी

Built for GDG India / Hack2Skill — *Build with AI: Code for Communities*, Track 1: AI for Digital Public Infrastructure & Governance (BRICS theme: Innovation) **Live prototype:** <https://nagarvaani-2i2l.onrender.com> **Source code:** <https://github.com/FalguniTakawale/NagarVaani> **Built by:** Falguni Takawale — one developer, using AI coding assistants as tools (see §14)

1. In one paragraph

NagarVaani is a civic-intelligence layer between citizens and government. People report local infrastructure problems — by web form, voice note or Telegram, in their own language — and the platform ranks every report on **how dangerous and urgent the problem really is** (safety, season, people affected, whether other areas report the same thing), **not on how many people clicked "upvote"**. Officials get a ranked queue scoped to their jurisdiction, AI-written briefs, a live hotspot map and a data-driven list of where demand is highest. Votes, comments and "same issue in my area" links are used for something different: **accountability and transparency** — showing everyone how often the same problem recurs and whether it was ever fixed.

Low votes ≠ low importance. A village of fifty with one contaminated handpump must outrank a busy road with a hundred upvotes about potholes.

2. The problem

Governments struggle to consolidate citizen feedback and align it with infrastructure priorities. Development requests live in fragmented places — helplines, letters, WhatsApp groups, social media, separate portals — so:

- Public spending can be **misaligned** with what people actually face.
- **Infrastructure gaps go unaddressed**, especially where people have little digital reach.
- There is **no way to measure** whether an initiative actually changed anything.

Most ways to "rank" public complaints reward whoever is loudest: the most votes, the largest city, the earliest post. That is exactly backwards for a village, a minority language group, or a problem people have stopped hoping will be fixed.

3. Our insight

1. **Urgency is a property of the problem, not of its audience.** Whether water is entering homes is independent of how many people saw the post.

2. **The same problem is usually repeated.** A blocked drain in one ward is often one drain line across five wards. Seeing that turns five complaints into one fix.
3. **Votes are still valuable — for a different job.** They show public attention, let people outside an area express solidarity, and let everyone audit whether issues get resolved. They must not decide *who gets help first*.
4. **People should be able to speak.** Reporting in Hindi, Marathi or Tamil — even Hindi typed in English letters ("Hinglish") — by voice or chat removes the biggest barrier.

4. The solution

How one complaint travels

1. **Intake** — web form, microphone voice note (speech-to-text), or the Telegram bot. Any language, optional photos and GPS.
2. **Filter** (AI) — political or communal framing is rejected with a suggested neutral rewording.
3. **Classify** (AI) — category, language (Hinglish is recognised as Hindi), English translation, location hint, "is someone likely to be hurt soon?".
4. **Score** (plain arithmetic, **no AI**) — the L1–L5 hierarchy below gives a 0–100 priority that can be explained line by line.
5. **Brief** (AI) — a two-to-three sentence official brief and one recommended action.
6. **Public thread** — comments, "same issue in my area" links, automatic detection of place names in comments (cross-area pattern), translation into the viewer's language, read-aloud for people who find reading hard.
7. **Action and accountability** — the responsible official updates the status; the citizen and everyone who voted are notified; the original reporter can **dispute** a "resolved" that isn't actually fixed.

The scoring system (L1–L5)

Level	Question it answers	Effect
L1 Safety	Could someone be hurt in the next few days?	Puts the complaint in the 90–99 band , ordered by severity; votes add at most +1
L2 Season	Is this worse right now? (monsoon drains, summer water/power, post-monsoon trees)	Multiplier $\times 1.0$ – $\times 2.0$, fixed when first scored
L3 Type	How much does this problem type affect health and daily life?	Water 28 · drainage 26 · electricity 22 · tree hazard 20 · corruption 18 · garbage 16 · road 14 · other 10
L4 People	How many people does it affect?	Log-scaled, up to +15 (currently a default population, see §9)
L5 Pattern	Do other areas report the same thing?	+5 per linked area, up to +20
Votes	How much public attention?	Gentle nudge $\times (1 + 0.1 \cdot \log_{10}(\text{votes}+1))$ for non-safety issues

$\text{severity} = \text{L1} + \text{L3} + \text{L4} + \text{L5} \rightarrow \text{score} = \min(100, \text{severity} \times \text{L2})$; safety risks use the 90–99 band instead; then the vote nudge. Every complaint page shows its own breakdown ("Why this scores 80"), and a plain-language page, *How scoring works (L1–L5)*, explains it to citizens.

Demonstration from the real scoring code (demo data, current season): a village handpump with 1 vote and a safety risk scores **98.7**; a road full of potholes with **104 votes** scores **40.6**.

Where votes and comments matter (accountability)

- **Similar issues across the whole city** — not only the same ward — each with its status, so a neighbourhood can see a problem is repeated and fix it together.
- **"See also across the country"** — the same problem type in other cities.
- **Resolution counts** — e.g. "In Pune: 4 reports of this kind — 1 resolved, 3 still unresolved."
- **Voters are notified** when an official marks the complaint resolved, and can say if it is not.

5. What makes it different

Typical public-complaint tools	NagarVaani
Rank by votes, recency or city size	Rank by severity × season × people × pattern , with votes as a small, bounded signal
One language, typed only	Voice and text, Hindi / Marathi / Tamil / English , Hinglish understood, on-demand translation both ways , read-aloud
A black-box "AI" score	Explainable arithmetic — the AI only reads and classifies; the ranking is deterministic and shown line by line
Complaint → inbox	Jurisdiction-scoped officials' queue, briefs, hotspot map, investment flags, scheme-linked project priorities
No follow-through	Status log, email/Telegram updates, dispute a false "resolved" , voter notification
Corruption mixed in with potholes	A separate, login-gated corruption channel, never in public feeds
Closed system	MIT licence , provider-agnostic (Gemini ↔ Claude; Whisper via Hugging Face / Groq / OpenAI), CSV export for planners

(The left column describes common patterns in complaint portals generally, not any single product.)

6. What we built

For citizens

Report (text, mic, Telegram, photos, 360° images, GPS with automatic area-name lookup) · anonymous or signed-in · track by ID · feeds (Home, Trending with pagination, Near Me with radius, heatmap and email alerts, My complaints, My votes) · vote, comment, "same issue in my area" · translate any complaint or comment · 🗣️ read-aloud · Share (native share sheet, WhatsApp, Telegram, X, Facebook, e-mail, copy link) · 🚩 Flag spam or abuse · dispute a false resolution · help chatbot grounded in real features · government schemes directory · "How scoring works" page.

For officials

Admin-approved accounts (work e-mail only) · ward / city / state / central levels · ranked **My Queue** · All / Resolved · **Hotspot map** · mark in-progress / resolved (only inside their own jurisdiction) · **Investment flags** · **Data-driven project priorities** (unresolved demand by state and problem type, joined with Census-2011 population and mapped to the real central scheme that would fund it) with **CSV download** · admin verification queue.

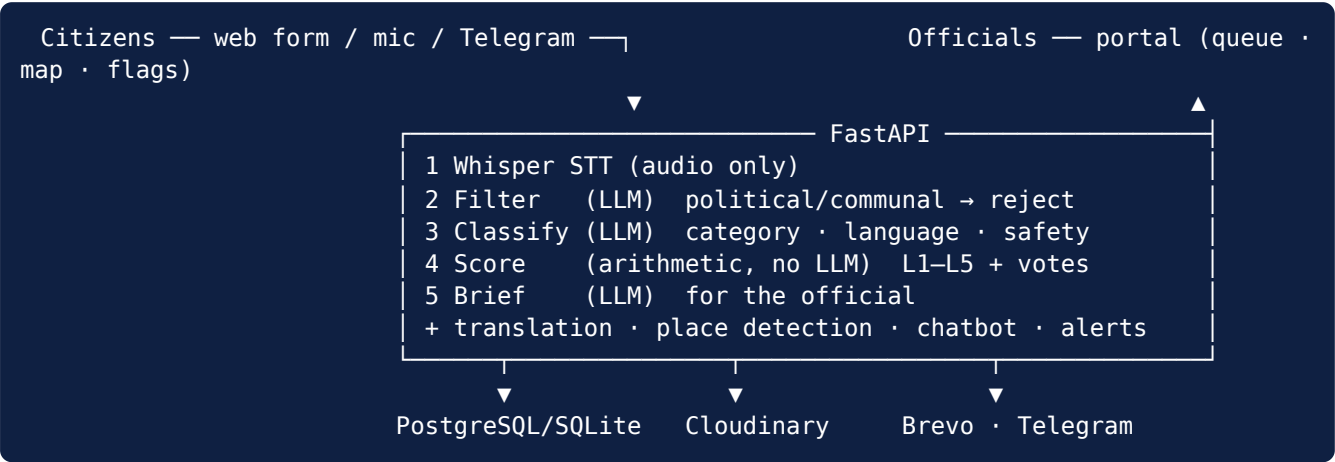
Platform

45 API operations across 9 routers · 11 database tables · e-mail OTP sign-up, password reset, optional Google sign-in · rate limits · security headers · Telegram webhook that registers itself on deploy · e-mail via Brevo (HTTPS) with SMTP fallback.

7. Technology

Layer	Choice	Notes
Backend	Python · FastAPI · SQLAlchemy (async)	One service serves the API and the frontend
Database	PostgreSQL (Neon) in production, SQLite locally	DATABASE_URL decides
Frontend	Vanilla JavaScript (ES modules), HTML, CSS	No framework, no build step; Leaflet + OpenStreetMap maps
Language AI	Google Gemini (default) · Claude switchable in one line	Filter, classify, translate, brief, place detection, chatbot
Speech-to-text	Whisper via Hugging Face (default), Groq or OpenAI	Language hint for Hindi / Marathi / Tamil
Speech output	Browser Web Speech API	No key, no server cost
Media	Cloudinary	Complaint photos / 360° images
Messaging	Telegram Bot API	Text and voice intake, account linking
E-mail	Brevo HTTPS API (SMTP fallback)	OTP, status updates, neighbourhood alerts
Auth	bcrypt · JWT · e-mail OTP · Google Identity (optional)	
Hosting	Render (web service) + Neon	One deploy, one URL

Architecture



8. Real, stub or demo — where things stand

✓ **Proper backend, working end to end** Intake (form, voice, Telegram) · the AI pipeline · L1–L5 scoring · votes, comments, cross-area links · status workflow with **jurisdiction enforcement** · disputes · e-mail and Telegram notifications (including to voters) · authentication and admin-approved officials · related/see-also queries · hotspot map and statistics (computed from the database) · project-priorities table and CSV ·

translation and read-aloud · rate limiting and security headers · citizen reports/flags stored for moderators (API).

🧩 Provision built, feature incomplete (a stub on purpose)

- **Population factor (L4)** uses a default of 10,000 per complaint. The `District` table exists; a per-area lookup is not wired in.
- **Infrastructure indices and public-investment plans** — not ingested. Only Census-2011 state populations (22 states/UTs) and real central-scheme names are used.
- **Project cost estimates** — deliberately *not* invented. Priorities give a triage index plus a suggested scheme, not a budget.
- **Moderation screen for citizen flags** — the API exists; there is no admin page yet.
- **Volunteer / NGO / partner forms** send e-mail to the team; nothing stores them.
- **Our Impact** — resolved-complaint counts are real; trees planted and recycled are "Coming soon".
- **Government schemes page** — real ministry links, but a fixed list, not a live feed.

👤 Demo or sample content (labelled in the app)

- `seed_data.py` demo users and sample complaints — for local demos only, never seed production.
- "Supporters" / partner cards and events on the Get Involved page — placeholders, marked DEMO.
- Static example cards shown before live data loads; example map pins for signed-out visitors only.

❌ **Not built** WhatsApp and SMS intake · countries other than India · installable / offline mode · automatic duplicate merging.

9. Fit to the challenge

Challenge requirement	Status
Voice, text and messaging-app intake	✅ web, mic, Telegram — ❌ WhatsApp
Multilingual	✅ four UI languages, AI understands many more; two-way translation; Hinglish
Aggregate and analyse	✅ scoring, cross-area patterns, city-wide and national "see also"
Combine with demographic data	🟡 Census-2011 state populations in the priorities table; per-complaint density is still a constant
Infrastructure indices, investment plans	❌ not yet — the largest gap
Surface demand hotspots	✅ live map, ranked priorities, CSV
Recommend priority projects	🟡 transparent triage index + matching central scheme; no costs
Measure impact	🟡 status log, disputes, resolved counts; no outcome dashboard yet
Digital Public Good	🟡 MIT licence, privacy note, provider-agnostic; open-data export pending
BRICS	❌ India only; the scoring, translation and dashboards are country-agnostic, the data layer is not

Verdict: a working, honest prototype of *citizen voice* → *explainable prioritisation* → *policymaker view*. The two additions that would make it convincing for the full brief are **one real infrastructure dataset** and a **second country's data pack** — both are data work, not re-architecture.

10. Quality, testing and security

Testing done

- **76 scripted endpoint checks** (public reads, auth gating, role and jurisdiction rules, duplicate and invalid input, rate limits, services with no keys) — all passing on the final build.
- **All 16 pages** loaded in a real browser at desktop and 390-pixel phone widths: no JavaScript errors, no sideways scrolling.
- Targeted browser scenarios for read-aloud (six voice/translation cases), pagination, GPS area lookup (four cases), and the ward-less-citizen dashboard.

Security review — what it found and fixed

Severity	Finding	Fix
Critical	E-mail verification endpoint returned a login for any already-verified account, and the complaint API exposed the author's user id — together allowing account takeover	Endpoint refuses verified accounts; API returns only an <code>is_author</code> flag
High	No brute-force limits on login or the 6-digit OTP	Per-account and per-IP limits (HTTP 429)
High	Any approved official could change any complaint's status	Status changes enforced inside the official's jurisdiction
Medium	Anyone could forge an official flag by typing a reserved prefix	Prefix reserved for officials; signed-in users can't post under another name
Medium	Image URLs could carry <code>javascript:</code> ; inline JS built from data	https-only URLs, inline handlers removed, <code>escapeHtml</code> escapes quotes
Medium	Anonymous translation/comment endpoints could burn AI credit	Per-IP limits
Low	No input caps, no security headers	Length caps; nosniff, frame-deny, referrer, permissions, HSTS

Known remaining risks — token kept in `localStorage`; rate limiter trusts `X-Forwarded-For` and lives in process memory (needs Redis for multi-instance); no Content-Security-Policy yet (inline handlers); AI filter **fails open** if the AI service is down (availability over strictness); seeded demo accounts must never reach a public deployment. Email OTP can be disabled for demo hosts with `REQUIRE_EMAIL_VERIFICATION=false`.

11. Where it can be expanded

The design already leaves room for these; none needs re-architecture.

1. **Real datasets** — a district-level population table feeding L4; infrastructure index and investment-plan tables joined into the priorities view (an "investment gap" = demand ÷ planned investment).
2. **More BRICS countries** — add a `country` field; a data pack (states, schemes, languages) per country; UI strings per locale. Scoring, translation and dashboards carry over unchanged.
3. **More channels** — WhatsApp Cloud API (clone the Telegram webhook), SMS/IVR for feature-phone users.
4. **Costed recommendations** — unit-cost tables from official schedules of rates, plus an optimiser.
5. **Impact measurement** — median time-to-resolve, dispute rate, oldest unresolved critical issue, and a "neglect index" (severity × days unresolved) per ward.

6. **Moderation UI and duplicate merging** — review screen for flags; embedding-based duplicate detection at submit.
7. **Digital Public Good** — open-data export (aggregated, anonymised), DPG-standard documentation.
8. **Reach** — installable / offline web app for weak networks; more UI languages.
9. **Hardening** — Redis rate limiting, CSP, httpOnly cookies, human-review queue when the AI is down.

12. Roadmap

Now (done)	Next	Later
Intake, scoring, official portal, hotspots, priorities + CSV, translation, read-aloud, security fixes, deploy	Real population + one infrastructure dataset; impact dashboard; moderation UI; PWA	WhatsApp/SMS, second country pack, costed recommendations, open-data portal

13. Running and deploying

See the [README](#): environment variables, local quick start, Render deployment, API reference. Optional features switch on when their key is present (AI, speech, photos, Telegram, Google sign-in, e-mail) — without a key each one degrades with a clear message rather than breaking the app.

14. How it was built

NagarVaani was conceived, specified and driven by one developer, **Falguni Takawale**: the problem framing, the scoring philosophy (severity and season over votes, votes for accountability), what to build and what to leave out, how to deploy it, and the hands-on testing of every flow. AI coding assistants were used as tools to write and review code faster — the way a developer uses a compiler or a linter — with the developer setting the requirements, rejecting what did not fit, and verifying the results.

Two decisions show that judgement at work: an early draft displayed a made-up ₹ cost for each project — it was **removed rather than replaced with another invented number**; and a security review of the finished system found a critical account-takeover path that was **fixed before submission**. The result is a deployed, tested system with an explicit list of what is real, what is a stub and what is not built — not a research write-up.

Independent civic-tech prototype. Not affiliated with any government body. MIT licence.